

Connect4 Student Folder

This folder is self-contained. Use the provided environment, evaluation helpers, scripted opponents, and CLI tools, then implement your own DQN and PPO agents in the starter skeletons.

Note that you can code completely from scratch, and also build your own Connect4 environment if you wish. However, here we are providing a Connect4 environment with some skeleton code for DQN and PPO learning; your code will need to be structured in a particular way to work with the provided environment, evaluation, and opponents scripts if you make use of them. That is what the `dqn.py` and `ppo.py` starter code files are there for.

Included Files

- `connect4/env.py`: fixed game environment and reward structure
- `connect4/evaluate.py`: gameplay and evaluation helpers
- `connect4/opponents.py`: random, heuristic, and Final Boss opponents
- `connect4/dqn.py`: DQN starter skeleton code
- `connect4/ppo.py`: PPO starter skeleton code
- `connect4/strong_dqn.py`: inference-only loader for the provided Final Boss checkpoint
- `play_connect4.py`: terminal play against humans, scripted agents, saved checkpoints, or `final-boss`
- `benchmark_connect4.py`: benchmark scripted agents and the Final Boss
- `checkpoints/finalboss.pt`: the trained model used as the `final-boss`

You're Up

You should implement your RL agents by taking care of the missing parts in `connect4/dqn.py` and `connect4/ppo.py`. They already contain some pieces, like masking legal moves and a wrapper to use the Connect4 environment properly.

- Define a PyTorch network for DQN and a policy/value network for PPO.
- Implement the main training function for each method.
- Implement checkpoint save/load helpers so your trained agent can be evaluated later.
- Implement an agent wrapper with a `select_action(env)` method so your model works with `evaluate.py` and `play_connect4.py`.

The ? placeholders in the config dataclasses are intentional. Replace them with your own hyperparameter choices before training.

Notes

- The observation is always from the current player's point of view.
- The action space is the set of columns, not board coordinates.
- The environment already applies gravity and checks wins for you.
- `play_connect4.py` is mainly useful after your checkpoint loading path works.
- `benchmark_connect4.py` is mainly for scripted opponents and the Final Boss, not for training your model.
- The environment gives small rewards for connecting two and connecting three. I (PK) found this helpful for early training, so kept it in the environment I am providing you. Feel free to change this as you see fit; in the end, the only thing that matters is your agent winning.

Getting Started

If you are not sure how to begin, here is a suggested sequence.

1. Read `connect4/env.py` and make sure you understand `reset()`, `step()`, `legal_actions_mask()`, and the observation format.
2. Replace the ? hyperparameter placeholders in `connect4/dqn.py` or `connect4/ppo.py` with your first reasonable choices.
3. Implement only the network and action-selection path first.
4. Check that your agent never selects an illegal column.
5. Implement one optimization step and make sure the loss changes.
6. Only after that, write the full training loop.

Minimal Workflow

The intended workflow is:

1. Build an environment with `Connect4Env(Connect4Config(...))`.
2. Use `env.get_observation()` and `env.legal_actions_mask()` as inputs to your model or policy.
3. Train against a provided opponent such as `RandomAgent` or `build_agent("random")`.
4. Evaluate with `evaluate_agent_pair(...)`.
5. Save a checkpoint, reload it with `from_checkpoint(...)`, and test it in `play_connect4.py`.

Your `train_dqn(...)` and `train_ppo(...)` functions do not need to match any hidden grader API, but they should return enough information for you to inspect training, such as the trained model, a history/log list, and the device used.

Dependencies

You need Python with at least:

- numpy
- torch

Usage

After you implement `DQNPolicyAgent.from_checkpoint`, you can play against your saved DQN checkpoint:

```
python play_connect4.py \  
  --agent-type dqn \  
  --agent-checkpoint checkpoints/connect4_dqn.pt
```

After you implement `PPOPolicyAgent.from_checkpoint`, you can play against your saved PPO checkpoint:

```
python play_connect4.py \  
  --agent-type ppo \  
  --agent-checkpoint checkpoints/connect4_ppo.pt
```

Play against The Final Boss:

```
python play_connect4.py \  
  --player1 human \  
  --player2 final-boss
```

Benchmark The Final Boss against the heuristic agent:

```
python benchmark_connect4.py \  
  --player1 final-boss \  
  --player2 heuristic \  
  --games 100
```